

```
"""
```

```
Fetch every active source's feed, parse entries, tag them, and store
new ones in the pool. Safe to re-run on a schedule (e.g. every few
hours via cron) -- duplicates are skipped via url_hash.
```

```
Usage:
```

```
python ingest.py                # ingest all sources in seed_sources.json
python ingest.py --check-dead    # also sweep existing items for dead links
```

```
Requires only the standard library, so it runs anywhere without pip
installs. Swap urllib/ElementTree for `feedparser` later if you want
broader feed-format compatibility -- it handles more edge cases than
this minimal parser.
```

```
"""
```

```
import argparse
import json
import sys
import urllib.request
import xml.etree.ElementTree as ET
from urllib.error import URLError, HTTPError

from db import init_db, get_conn, upsert_source, insert_item
from tagger import tag_item

USER_AGENT = "StumbleBot/0.1 (+content discovery ingester)"
TIMEOUT_SECS = 10

ATOM_NS = "{http://www.w3.org/2005/Atom}"
```

```
def fetch(url: str) -> bytes:
    req = urllib.request.Request(url, headers={"User-Agent": USER_AGENT})
    with urllib.request.urlopen(req, timeout=TIMEOUT_SECS) as resp:
        return resp.read()
```

```
def parse_feed(raw: bytes) -> list[dict]:
    """Handles basic RSS 2.0 <item> and Atom <entry> formats."""
    root = ET.fromstring(raw)
    entries = []

    # RSS 2.0
    for item in root.iter("item"):
```

```

        entries.append({
            "title": (item.findtext("title") or "").strip(),
            "url": (item.findtext("link") or "").strip(),
            "excerpt": (item.findtext("description") or "").strip(),
            "published_at": (item.findtext("pubDate") or "").strip() or None,
        })

# Atom
for entry in root.iter(f"{ATOM_NS}entry"):
    link_el = entry.find(f"{ATOM_NS}link")
    entries.append({
        "title": (entry.findtext(f"{ATOM_NS}title") or "").strip(),
        "url": link_el.get("href") if link_el is not None else "",
        "excerpt": (entry.findtext(f"{ATOM_NS}summary") or "").strip(),
        "published_at": (entry.findtext(f"{ATOM_NS}updated") or "").strip() or None
    })

return [e for e in entries if e["url"] and e["title"]]

def ingest_source(conn, source_row: dict) -> tuple[int, int]:
    """Returns (new_items, skipped_duplicates) for one source."""
    source_id = upsert_source(
        conn, source_row["name"], source_row["site_url"],
        source_row["feed_url"], source_row["category"],
    )

    try:
        raw = fetch(source_row["feed_url"])
        entries = parse_feed(raw)
    except (URLLError, HTTPError, ET.ParseError) as e:
        print(f" [warn] could not fetch/parse {source_row['name']}: {e}", file=sys.stderr)
        return 0, 0

    new_count, dup_count = 0, 0
    for entry in entries:
        tags = tag_item(source_row["category"], entry["title"], entry["excerpt"])
        inserted = insert_item(
            conn, source_id, entry["url"], entry["title"],
            entry["excerpt"][:400], # keep excerpts short -- this is a discovery
            image_url=None, # TODO: extract og:image on first click-thru
            tags=tags, published_at=entry["published_at"],
        )
        if inserted:
            new_count += 1
        else:
            dup_count += 1

```

```

return new_count, dup_count

def check_dead_links(conn, limit: int = 200):
    """Sweep a batch of existing items and flag ones that 404/error."""
    rows = conn.execute(
        "SELECT id, url FROM items WHERE is_dead = 0 ORDER BY ingested_at ASC LIM
        (limit,)",
    ).fetchall()

    flagged = 0
    for row in rows:
        try:
            req = urllib.request.Request(row["url"], method="HEAD",
                                         headers={"User-Agent": USER_AGENT})
            urllib.request.urlopen(req, timeout=TIMEOUT_SECS)
        except HTTPError as e:
            if e.code >= 400:
                conn.execute("UPDATE items SET is_dead = 1 WHERE id = ?", (row["i
                flagged += 1
        except URLError:
            conn.execute("UPDATE items SET is_dead = 1 WHERE id = ?", (row["id"],
            flagged += 1

    return flagged

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--sources", default="seed_sources.json")
    parser.add_argument("--check-dead", action="store_true")
    args = parser.parse_args()

    init_db()

    with open(args.sources) as f:
        sources = json.load(f)

    total_new, total_dup = 0, 0
    with get_conn() as conn:
        for source_row in sources:
            print(f"Ingesting {source_row['name']}...")
            new_count, dup_count = ingest_source(conn, source_row)
            total_new += new_count
            total_dup += dup_count
            print(f"  {new_count} new, {dup_count} already seen")

```

```
    if args.check_dead:
        print("Sweeping for dead links...")
        flagged = check_dead_links(conn)
        print(f"    flagged {flagged} dead links")

    print(f"\nDone. {total_new} new items added, {total_dup} duplicates skipped.")

if __name__ == "__main__":
    main()
```